

Worked Example: Build a Huffman Tree

Given an image: 8*8 8-bit grayscale image

58	61	67	72	72	79	85	79
52	58	61	67	72	79	85	91
58	61	67	72	67	79	85	91
61	67	72	79	72	85	79	85
52	58	61	67	72	79	85	91
58	61	67	72	79	72	85	91
52	58	61	67	72	79	85	91
52	58	61	67	72	79	85	91

Pixel values Frequency

Gray level	Frequency
52	4
58	7
61	8
67	10
72	11
79	10
85	9
91	5

What to remember

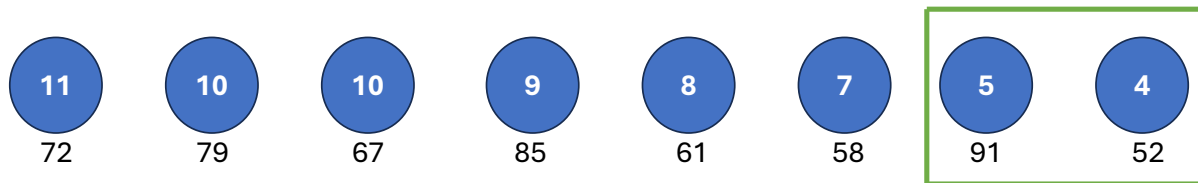
Total Number of pixel in the image: $8 \times 8 = 64$. [Original Storage: (Total no. of pixel * bit) = $64 \times 8 = 512$ bits]

Sum of the frequency: $4+7+8+10+11+10+9+5=64$

Since, the total number of pixel and sum of the frequency are equal. Hence, the histogram is correct

Worked Example: Build a Huffman Tree

- Sort the frequency of pixel values in descending order to construct the Huffman tree
- Combine the two nodes with smallest frequency into a new parent node whose frequency is the sum
- Rearrange the nodes after making a new node

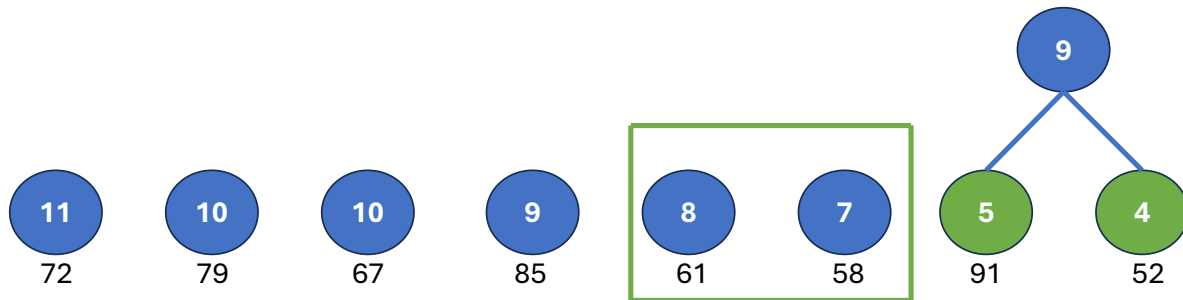


Note:

Since 4 and 5 are the two lowest frequencies, we combine them to form a new parent node. The value of the parent node is the sum of these two values: 9.

Worked Example: Build a Huffman Tree

- Sort the frequency of pixel values in descending order to construct the Huffman tree
- Combine the two nodes with smallest frequency into a new parent node whose frequency is the sum
- Rearrange the nodes after making a new node

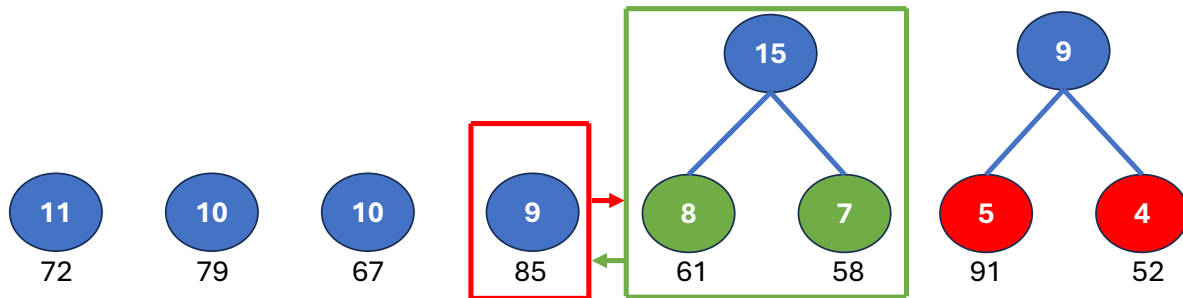


Note:

Now, lowest two nodes are 7 and 8. So, make a new parent nodes: $7+8=15$

Worked Example: Build a Huffman Tree

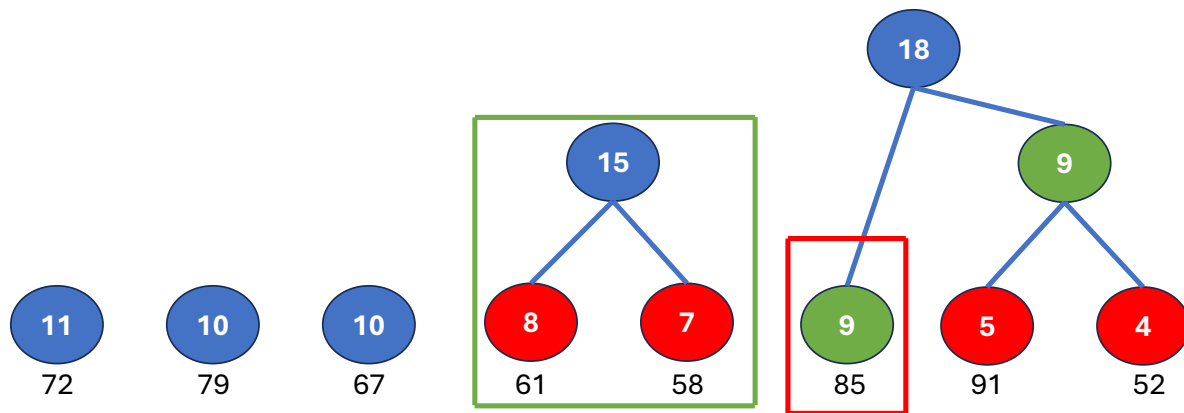
- Sort the frequency of pixel values in descending order to construct the Huffman tree
- Combine the two nodes with smallest frequency into a new parent node whose frequency is the sum
- Rearrange the nodes after making a new node



Note:

Since 9 and 9 are now the smallest nodes, we place them side by side for convenience so they can be merged into a new parent node.

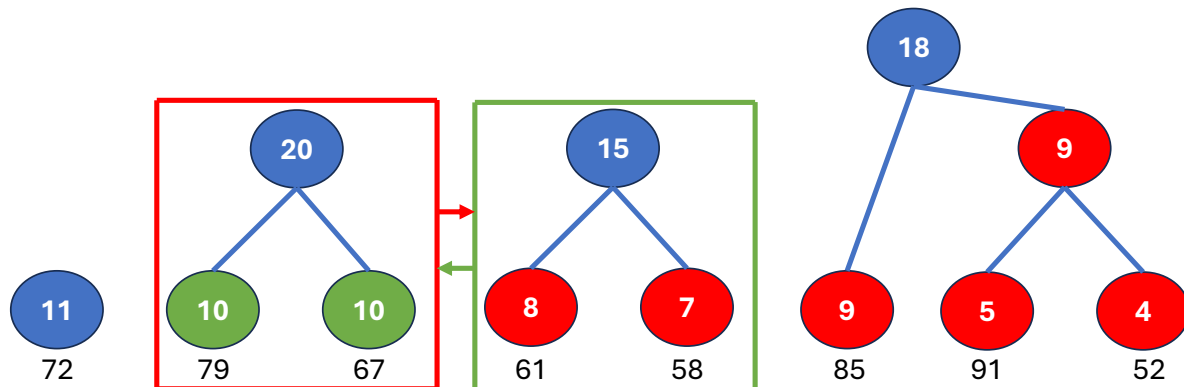
Worked Example: Build a Huffman Tree



Note:

Among 11, 10, 10, 15, and 18; node 10 and 10 are the smallest.

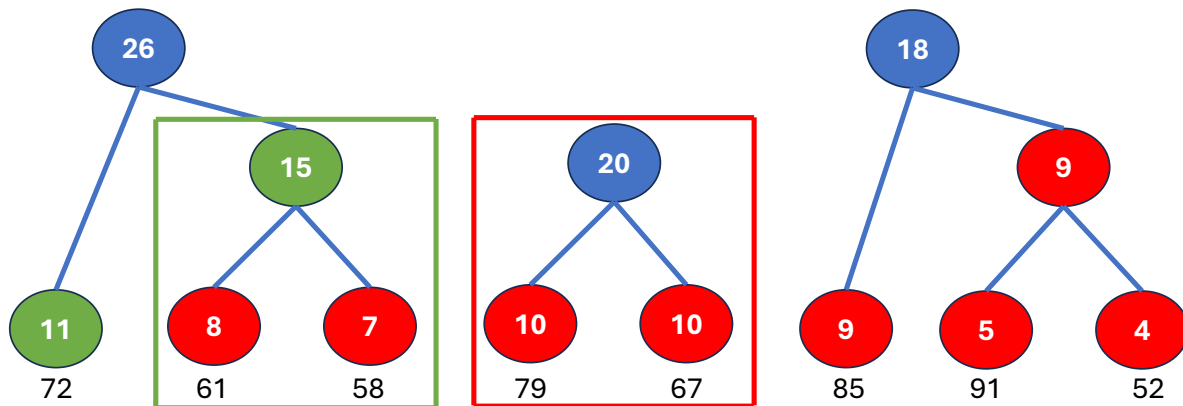
Worked Example: Build a Huffman Tree



Note:

Again, 11 and 15 are now the smallest nodes. So, we again place them side by side for convenience so they can be merged into a new parent node.

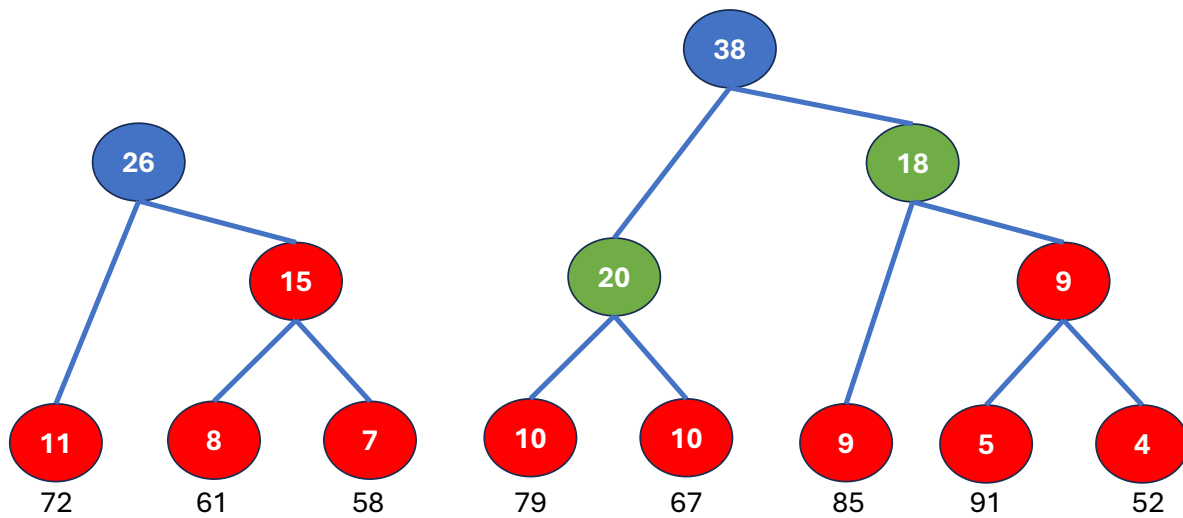
Worked Example: Build a Huffman Tree



Note:

Merge 18 and 20 as these two are now the smallest two nodes.

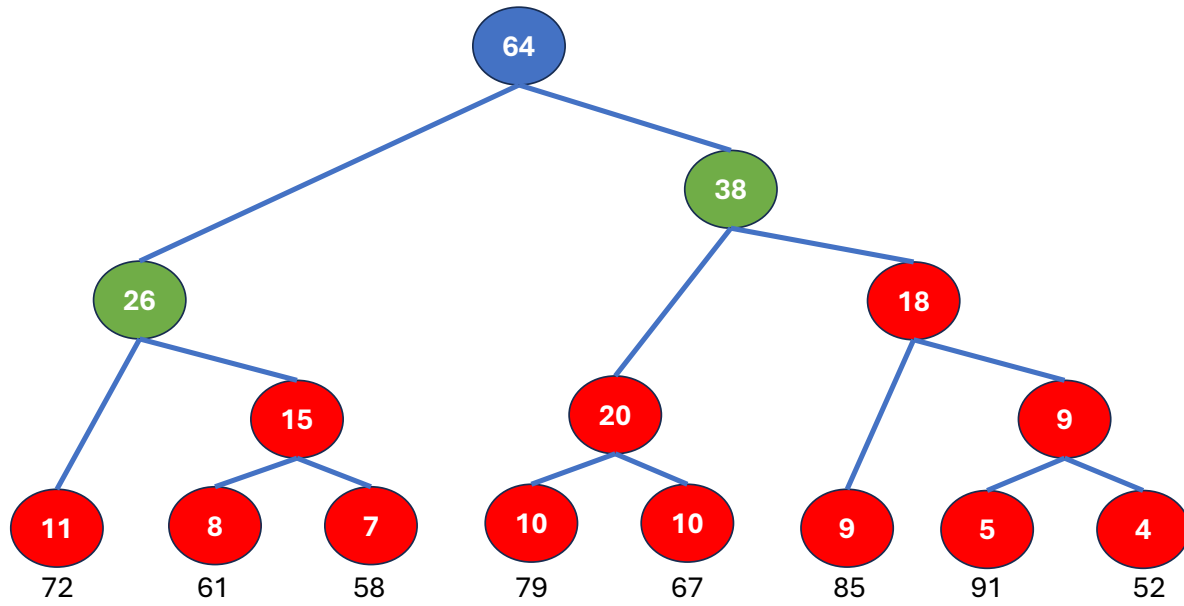
Worked Example: Build a Huffman Tree



Note:

Finally, merge 26 and 38 to form the root node, whose value is 64. This matches the total sum of all frequencies, which is 64. Since the image size is 8×8 , the total number of pixels is also 64.

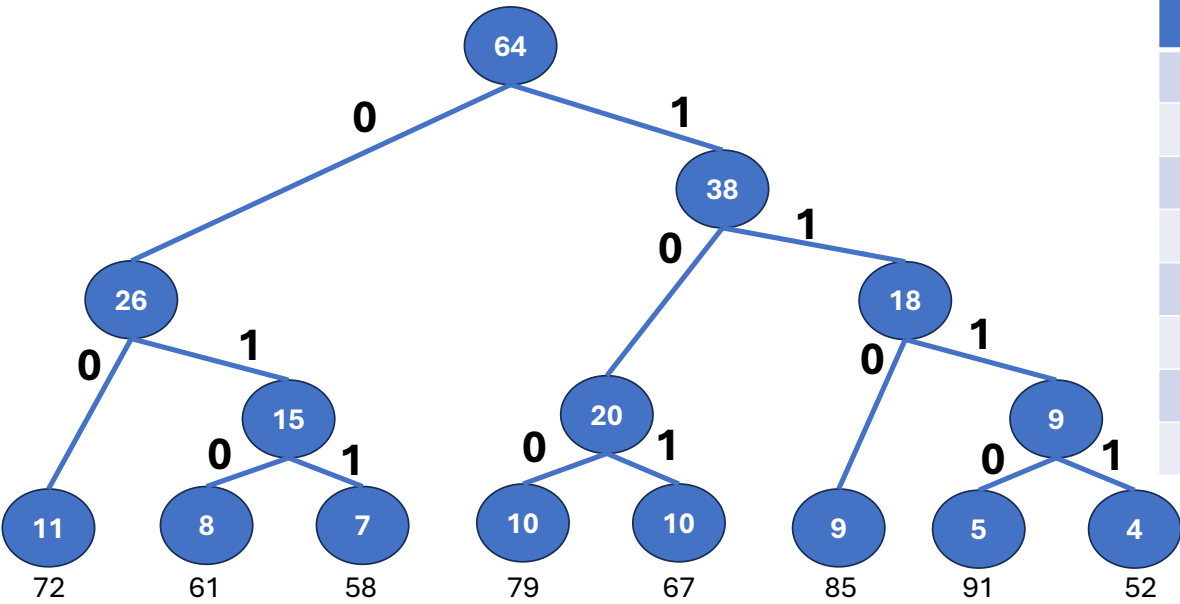
Worked Example: Build a Huffman Tree



Note:

Now assign 0 to the left edge and 1 to the right edge.

Worked Example: Build a Huffman Tree



Pixel Values	Codes	Bits	Freq.	Total (bit size x freq)
72	00	2	11	22
61	010	3	8	24
58	011	3	7	21
79	100	3	10	30
67	101	3	10	30
85	110	3	9	27
91	1110	4	5	20
52	1111	4	4	16

Note:
 After applying Huffman coding, the size of the image becomes: $22+24+21+30+30+27+20+16 = 190$ bits

Practice Problem: Solve Huffman Coding by Hand

Problem

A 3x3 10-bit Grayscale image:

52	52	52	55
52	55	55	61
52	52	61	61
55	52	61	70

Tasks:

1. Draw the Huffman tree.
2. Write one valid code for each pixel.
3. Compute average bits/pixel.
4. Compare with fixed-length coding.

Answer outline

Possible combining order:

$$4+1=5$$

$$5+4=9$$

$$9+7=16 \text{ (Root)}$$

One valid code set:

$$52=0, 55=10, 61=110, 70=111$$

Lengths: 1,2,3,3

$$\begin{aligned} \text{Avg bits/pixel} &= \frac{(1 \times 7) + (2 \times 4) + (3 \times 4) + (3 \times 1)}{16} \\ &= \frac{30}{16} = 1.875 \text{ bits/pixel} \approx 2 \text{ bits/pixel} \end{aligned}$$

Fixed-length = 10 bit/pixel